

SHL Assessment Recommendation Assistant — Approach Document

Author: Sanket Shakya

1. Solution Overview

The objective of this project was to build a conversational AI assistant that recommends the most relevant SHL assessments based on hiring requirements or job descriptions. The system uses a Retrieval-Augmented Generation (RAG) architecture, where semantic search retrieves relevant assessments from the SHL catalog before an LLM generates grounded explanations, reducing hallucinations and ensuring recommendations are limited to official SHL assessments.

The solution consists of a FastAPI backend, Qdrant vector database, Sentence Transformers for semantic retrieval, Gemini 2.5 Flash for reasoning, and a Streamlit interface for demonstration.

2. Design Choices

- **FastAPI** — Lightweight REST API framework for /health and /chat.
- **Sentence Transformers (all-MiniLM-L6-v2)** — Generates semantic embeddings for assessments and user queries.
- **Qdrant** — Stores assessment embeddings; performs efficient cosine similarity search.
- **Gemini 2.5 Flash** — Produces natural language explanations while remaining grounded in retrieved assessments.
- **Streamlit** — Simple interactive interface for demonstrating the assistant.

A RAG architecture was chosen instead of direct LLM prompting because recommendations must always originate from the official SHL catalog.

3. Retrieval Setup

The SHL assessment catalog was converted into structured JSON containing metadata: name, description, categories, job levels, duration, remote testing support, adaptive support, and official SHL URL. Each assessment is embedded using Sentence Transformers and indexed in Qdrant.

On a hiring requirement: query embedded → Qdrant retrieves most semantically similar assessments → retrieval scores attached → results passed to Gemini for explanation. This enables semantic matching instead of relying solely on keyword search.

4. Prompt Design

Gemini receives conversation history, latest user request, retrieved SHL assessments, and retrieval scores. It is explicitly instructed to:

- Use only retrieved assessments.
- Never invent assessment names or URLs.
- Explain why each retrieved assessment matches the hiring requirement.
- Prefer assessments with higher retrieval scores when appropriate.

Gemini returns structured JSON containing only the recommendation summary and explanations; assessment metadata is supplied directly by the backend.

5. Conversation Handling

- **Clarification** — Requests additional information for vague queries.
- **Recommendation** — Suggests relevant SHL assessments for hiring requirements.
- **Refinement** — Uses the complete conversation history to refine previous recommendations.
- **Comparison** — Compares two SHL assessments using catalog information.

- **Out-of-Scope Handling** — Politely declines requests unrelated to SHL assessments.

The API remains stateless by requiring the client to send the complete conversation history with each request.

6. Evaluation

The system was evaluated using representative hiring scenarios: Java Backend Developer, Python Developer, Data Scientist, Sales Executive, and Graduate Hiring. Additional conversational testing included clarification requests, recommendation refinement, assessment comparison, and out-of-scope queries.

Evaluation focused on recommendation relevance, grounding to the SHL catalog, conversational correctness, and consistency of ranking using retrieval scores.

7. Challenges & Future Improvements

The primary challenge was preventing LLM hallucinations, addressed by restricting Gemini to retrieved assessments and letting the backend supply all metadata. Recommendation quality was further improved by incorporating retrieval scores into the ranking process.

Future improvements: hybrid keyword + semantic retrieval, cross-encoder reranking, improved comparison matching, automated evaluation metrics, and streaming responses.

8. AI Tools Used

- **ChatGPT (OpenAI)** — Architecture discussions, debugging, prompt refinement, implementation guidance.
- **Google Gemini 2.5 Flash** — Recommendation generation and assessment comparison.
- **Sentence Transformers** — Semantic embedding generation.
- **Qdrant** — Vector similarity search and retrieval.

Architecture Diagram

